## Exercise 4.1: Traditional Testing vs. ML Model Testing

**Name at least four differences between testing traditional software and testing a machine learning model.**

Traditional software is logic-driven, humans write the rules, and testing checks that those rules behave as intended. ML systems are data-driven , the logic is learned from examples, so testing has to work very differently. Four key differences:

**1. Perfect pass/fail vs. tolerance for errors** In traditional software, the goal is all tests should pass. A unit test either passes or fails, and a failure means there is a bug to fix. With ML models you almost always have some errors; there is no such thing as 100% accuracy on real data.

**2. Deterministic logic vs. statistical behaviour** Traditional code behaves the same way every time for the same input. ML models can produce different outputs for inputs that look nearly identical. Even a tiny perturbation to an image can flip a classifier's prediction.

**3. Code coverage vs. data representativeness** Traditional testing aims for good code coverage  making sure every branch and function gets exercised. ML testing has no equivalent. Instead, the question is whether the test set is representative of the real deployment distribution, and whether it's large enough to give meaningful statistics. Coverage of the input space is practically infinite and can never be fully tested.

**4. Stable behaviour vs. data drift** Once traditional software passes its tests, its behaviour is fixed. ML models face a problem called **data drift**- the real-world distribution of inputs can shift after deployment (e.g., weather changes, new traffic patterns in CARLA), causing model performance to silently degrade even though the model itself hasnot changed. This makes ongoing monitoring essential in ML, whereas traditional software usually just needs regression tests to catch regressions from code changes.

## Exercise 4.4: Distribution Shift Types

**Consider the following three scenarios involving the CARLA autonomous driving model**
**(trained on sunny daytime data):**
**1. The model is deployed during winter, when roads are often wet and the sun is at a**
**low angle, creating glare in camera images.**
**2. A new city zone is added to the test route where 60 % of road users are cyclists - a class that was rare (< 5 %) in the original training data.**
**3. The city replaces all old traffic light housings with a new, slimmer design that the model has never seen.**
**For each scenario:**
**a) Identify which type of distribution shift applies (covariate shift, label shift, or concept shift). Justify your answer.**
**b) Explain what effect you would expect on model performance.,**

**c) Suggest one mitigation strategy.**

Exercise 4.4 — Distribution Shift Types

Covariate shift: the input distribution P(X) changes, but the relationship P(Y|X) stays the same — the world hasnot changed, just what the camera sees

Label shift: the class proportions P(Y) change — certain things appear more or less frequently

Concept shift: the relationship P(Y|X) itself changes — what the inputs *mean* for classification has fundamentally changed

---

Scenario 1 — Winter weather: wet roads and low-angle sun glare

a)  Covariate Shift

The input images look different — glare, reflections off wet tarmac, different colour temperature. But the underlying truth about what constitutes a pedestrian or vehicle hasn't changed at all. Only P(X) has shifted because the camera is now receiving very different pixel distributions than during sunny daytime training.

b) Expected effect on performance

The model will likely show degraded detection rates across all classes, but particularly for pedestrians and vehicles in high-glare regions of the image. The features the model learned to rely on — shadow patterns, colour contrasts, sharp edges — will be disrupted by lens flare and wet-road reflections. Confidence scores may also drop, making threshold-based decisions less reliable.

c) Mitigation

Augment the CARLA training data with simulated adverse lighting conditions, glare overlays, rain effects, low sun angles

---

Scenario 2 — New city zone with 60% cyclists (previously < 5%)

a) Type: Label Shift

The input images are not  different — it's still daytime, same camera, same city-style rendering. the proportion of classes has changed: cyclists now dominate road users rather than being rare

cases.  cyclists went from a negligible fraction to the majority class. a cyclist frame still looks like a cyclist frame — but the frequency with which that class appears has moved dramatically.

b) Expected effect on performance

Because cyclists were severely underrepresented during training (< 5%), the model has seen very few examples of them and will have poor recall on that class.  The model is tuned to expect mostly pedestrians and vehicles. In the new zone it will frequently misclassify cyclists or fail to detect them at all. Rare classes in training become performance sinkholes at deployment when those classes suddenly dominate.

c) Mitigation

Re-balance the training data by oversampling cyclist scenarios in CARLA, or apply class-weighted loss during training to give cyclist examples higher influence.

---

Scenario 3 — New slimmer traffic light housing design

a) Type: Concept Shift

This is the most dangerous of the three. The inputs (camera images) now contain traffic light housings the model has never seen — but more importantly. The visual features that previously mapped to "traffic light present" ,no longer match the new slim design. The model's learned concept of what a traffic light looks like is now wrong.

b) Expected effect on performance

The traffic light detector will have sharply degraded recall on the new housings — potentially treating them as absent altogether. the traffic-light model only detects presence, not state, so if it now also misses presence, the planner has zero information about traffic lights at intersections using the new design. The safety consequence is severe.

c) Mitigation

This requires a targeted data collection and retraining effort — render or photograph the new housing design, label it, and retrain or fine-tune the traffic light classifier. This is also a strong argument for ODD restriction in the interim: until the model is retrained, the new housing zones should sit outside the declared ODD, and the system should not be deployed there autonomously.

### Exercise 4.5: ODD Coverage with k-Projection Coverage

**You defined an ODD in Exercise Sheet 2. Now assess how well your test set covers this ODD using k-projection coverage.**

**1. Describe in your own words what k-projection coverage measures and why it is a useful metric for ODD coverage.**

**2. Download the implementation from https://github.com/kkirchheim/odd-coverage and compute the k-projection coverage of your test set for k ∈ {1, 2, 3}.**
**3. How does coverage change with increasing k? What does this tell you about the adequacy of your test set?**

**1. What k-projection coverage measures**

The core problem with ODD coverage is combinatorial explosion.ODD has 4 dimensions — weather, lighting, scene, speed — and requiring every possible combination to appear in the test set would demand hundreds of scenarios. k-projection coverage solves this. Instead of requiring all combinations, it only asks whether every k-way combination of dimension values appears at least once.

At k=1 you check each dimension value in isolation .At k=2 you check every pair .At k=3 every triple must co-occur somewhere. The higher the k, the stricter the requirement, and the more interaction failures the metric can detect.

It is useful because real-world failures almost always happen at the intersection of conditions, not in isolation. A model might handle fog fine and handle town01 fine, but fail when both occur together — and only k≥2 coverage would flag that gap.

**2. Results**

k=1 : 70.0% ( 7 / 10 bins covered)

k=2 : 41.7% (15 / 36 bins covered)

k=3 : 23.2% (13 / 56 bins covered)

3. k=1 → k=2 → k=3

70% → 42% → 23%

The steep drop at each step reveals that four test splits cover individual conditions reasonably well but almost no interactions between them. This is the consequence of having only four scenarios .The safety implication is significant. My models will face combined conditions in real deployment — a foggy night in town01, a rainy daytime in an unfamiliar scene — and your test set provides zero evidence about model behaviour under those combinations. The pedestrian model's already catastrophic recall of 0.000

on test-fog and test-town-01 individually suggests that combined OOD conditions would be even worse

**Exercise 4.6: Designing a Test Suite from Safety Constraints**

Refer back to Exercise 2.7 in Sheet 2, where you derived safety constraints from Unsafe Control Actions (UCAs). For each **model-level constraint**, design **one test case** using the following structure:

| Constraint ID | Constraint | Test input description | Expected output | Pass criterion |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |
| | | | | |

Hint: Include the constraint ID (e.g., SC-1, SC-3, SC-4) in the first column. This links your test design explicitly to the safety constraints you identified.

| Constraint ID | Constraint | Test input description | Expected output | Pass criterion |
|---|---|---|---|---|
| SC-1 | Pedestrian detector must achieve recall ≥ 99% on the in-distribution test set, so missed detections are rare enough for the planner to reliably trigger emergency braking | CARLA test split: sunny daytime images confirmed to contain a pedestrian (positive-class frames only), covering urban streets, intersections, varying pedestrian distances and occlusion levels | For each input, model outputs prediction = "pedestrian present" (positive) | Recall = TP / (TP + FN) ≥ 0.99 across all positive-class test frames |
| SC-2 | Pedestrian and vehicle models must achieve precision ≥ 90% to limit false alarms that would cause spurious emergency braking | CARLA test split: sunny daytime images confirmed to contain no pedestrian or vehicle (negative-class frames) — empty roads, static scenes, background-only | For each input, model outputs prediction = "not present" (negative) | Precision = TP / (TP + FP) ≥ 0.90; false positive rate stays low enough that debounce logic is not overwhelmed |
| SC-3 | Pedestrian model inference latency must not exceed 50 ms per frame (contributing to the 100 ms end-to-end | 500 consecutive CARLA frames fed to the pedestrian detector under realistic hardware load; no batching — single-frame | Each frame produces a prediction with a recorded | 99th-percentile latency ≤ 50 ms; no single frame exceeds 100 ms |

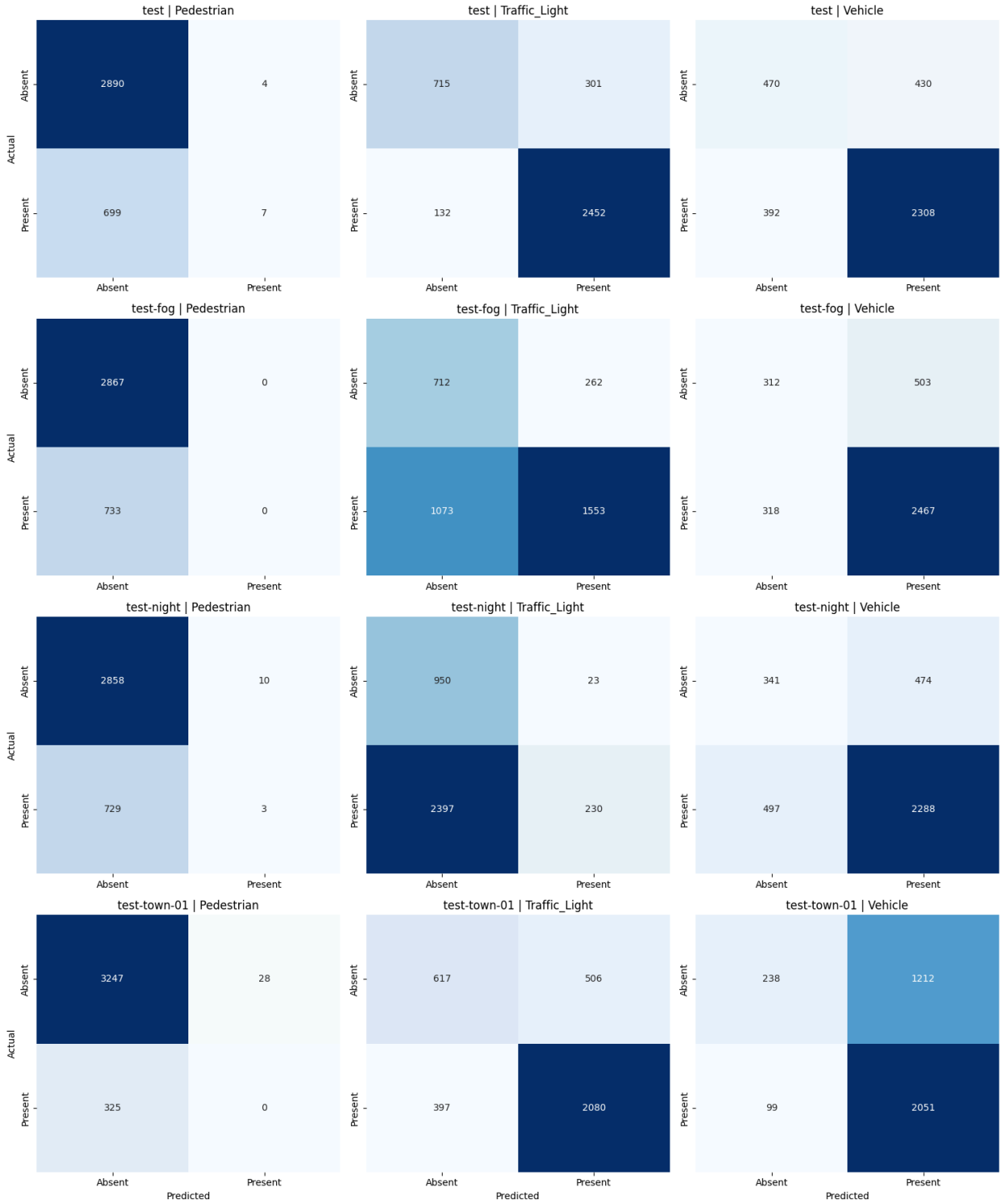
| | | | | |
|---|---|---|---|---|
| | pipeline budget) | inference as in deployment | wall-clock inference time | |

**Exercise 4.7: Per-Class Evaluation**
**Evaluate each of your three models on the provided test set.**
**1. Compute and report precision, recall, and F1-score for each model.**
**2. Plot a confusion matrix for each model.**
**3. Which model has the lowest recall? Is this the model you expected to be hardest based on your hazard analysis?**
**4. Based on a safety argument: what minimum recall would you require for the pedestrian model before considering deployment? Justify your threshold.**

1.

| split | Model | Precision | Recall | F1 |
|---|---|---|---|---|
| test | Pedestrian | 0.636 | 0.010 | 0.020 |
| test | Traffic Light | 0.891 | 0.949 | 0.919 |
| test | Vehicle | 0.843 | 0.855 | 0.849 |
| Test fog | Pedestrian | 0 | 0 | 0 |
| Test fog | Traffic Light | 0.856 | 0.591 | 0.699 |
| Test fog | Vehicle | 0.831 | 0.886 | 0.857 |
| Test night | Pedestrian | 0.231 | 0.004 | 0.008 |
| Test night | Traffic Light | 0.909 | 0.088 | 0.160 |
| Test night | Vehicle | 0.828 | 0.822 | 0.825 |
| Test town | Pedestrian | 0 | 0 | 0 |
| Test town | Traffic Light | 0.804 | 0.840 | 0.822 |
| Test town | Vehicle | 0.629 | 0.954 | 0.758 |

**2.**




**3.** The pedestrian model has the lowest recall . It is a complete failure . This is what the hazard analysis predicted. Also the traffic light model collapses at night

**4.** Minimum threshold recall > 0.99. The pedestrian detector is sole trigger for emergency braking in the module. The planner has no camera access, no redundant sensor, and no fallback. A false negative predicting absent when a pedestrian is present, leads to immediately and unconditionally to a missed brake command.

Address class imbalance in the data.